



TP0 – Déploiement de logiciel

Atelier logiciel : systèmes de transmission – SIT213

1 Objectifs

Dans vos activités de projets logiciels de l'année précédente (par exemple en FIP INF112), vous avez été amenés à livrer itérativement le fruit de votre travail, sous forme d'une archive logicielle. Malgré quelques exigences du client (par exemple, cahier de suivi, export Eclipse), votre processus de livraison/déploiement n'était pas formalisé, ce qui a pu entraîner quelques problèmes (livraison incomplète ou avec des versions de fichiers incorrectes).

Dans le cadre du projet SIT213, vous allez améliorer la **qualité** de ces processus. Comme vous allez utiliser un environnement de développement (typiquement votre ordinateur personnel) différent de celui de votre client (machine Linux de l'école), vous allez outiller la livraison. Ce premier TP doit vous permettre de prendre un bon départ.

2 Contenu

Pour cette séance, vous allez travailler dans l'*environnement du client* (machine Linux des salles de TP de l'école). Gardez en tête que chaque fois que vous livrerez une version de votre futur logiciel, le client sera dans ce même contexte de travail.

Exercice 1 (*Mise en jambe*)

Récupérez le script [maCommande](#) sur Moodle.

▷ Comment l'exécuter dans le shell ?

Exercice 2 (*Mise en forme de votre livrable*)

Vous allez travailler le contexte de livraison et déploiement d'un projet Java réduit à sa plus simple expression.

Voici un bref descriptif du travail de cette séance :

Créez une arborescence pour votre projet, avec un sous-répertoire `src` pour les fichiers `.java`, un sous-répertoire `bin` pour les `.class`, un sous-répertoire `docs` pour la javadoc.

Créez dans `src` un `HelloWorld.java` dont le `main()` affiche le très célèbre message `"Hello World !"`.

Vous allez créer des scripts pour aider votre futur client à déployer le logiciel. Ces scripts seront stockés à la racine de votre projet.

Positionnez-vous maintenant à cet endroit.

▷ Question 2.1 :

Quelle commande permet de compiler votre code et de placer les fichiers `.class` obtenus dans le sous-répertoire `bin` ?

Testez-la. Créez un script `compile` permettant de lancer cette commande.

▷ Question 2.2 :

Quelle commande permet de générer la javadoc et de la placer dans le sous-répertoire `docs` ? Créez un script `genDoc` pour réaliser cette action. NB : aurait-on pu nommer ce script `javadoc` ? Expliquez.

▷ Question 2.3 :

Quelle commande permet de lancer l'exécution de votre programme ? Créez un script `hello` pour le faire.

▷ Question 2.4 :

Comment effacer tous les `.class` et toute la javadoc ? Créez un script `cleanAll` pour le faire.

Exercice 3 (*Améliorations*)

Vous allez maintenant ajouter une chaîne de caractères comme paramètre du `main()`. Sans paramètre, le programme affichera « Hello World ! » (affichage par défaut), alors qu'avec paramètre, le programme affichera « Hello <toute la chaîne passée en paramètre> ! ».

▷ Question 3.1 :

Modifiez votre `HelloWorld` en conséquence. Créez un script `runTests` permettant de lancer plusieurs fois de suite votre programme, avec des paramètres d'appel différents.

▷ Question 3.2 :

Quelle commande permet de créer une archive de tous les fichiers/dossiers utiles de votre projet ?

Créez un script `genDeliverable` permettant de créer l'archive de votre livrable et dont le nom est composé des deux noms des auteurs (vous !) sous la forme `Nom1-Nom2.tar.gz`. Que peut-il se passer si vous exécutez plusieurs fois cette commande ? Essayez ! Cela n'empêche pas de prendre ses précautions : modifiez le `cleanAll` en conséquence !

▷ Question 3.3 :

Ajouter un fichier `readme` détaillant le contenu de l'archive à livrer.

Avant de générer cette archive pour la livrer, vous devez être sûrs de bien avoir la dernière version de votre projet : si vous utilisez Git pour le développement, la commande `pull` fera l'affaire !

Pour renforcer votre confiance dans votre livrable, il est aussi souhaitable de tester votre livrable directement, en dehors de l'environnement de développement habituel (IDE ou espace projet adapté par exemple).

Voici comment vous pourrez procéder :

1. Récupérer la version à jour du `src` du projet dans un espace de test prévu à cet effet.
2. Exécuter les divers tests d'avant livraison.
3. Nettoyer (effacer les fichiers étrangers au livrable).
4. Générer le livrable.

Exercice 4 (*Livraisons croisées*)

Vous avez donc maintenant de quoi produire votre livrable.

Celui-ci doit donc être un `tar.gz` dont le nom est de la forme `nom1-nom2.tar.gz` et il doit contenir :

- des répertoires `src`, `bin`, `docs` ; seul le répertoire `src` n'est pas vide (il doit contenir votre classe `HelloWorld.java`) ;
- les scripts `compile`, `genDoc`, `cleanAll`, `hello` et `runTests` ;
- un fichier `readme`.

Attention, votre script `genDeliverable`, qui fabrique votre livrable automatiquement, ne fait pas partie du livrable.

- ▷ **Envoyez votre livrable au binôme voisin, qui fera de même avec vous. Testez que tout est conforme et faites un compte-rendu aux auteurs.**

3 Pour aller plus loin

Des outils plus avancés que de simples scripts existent. Par exemple, l'outil `ant` (<http://ant.apache.org/>) peut être utilisé pour construire et déployer des logiciels Java. Vous pourrez avantageusement suivre le tutoriel suivant <https://ant.apache.org/manual/tutorial-HelloWorldWithAnt.html>.